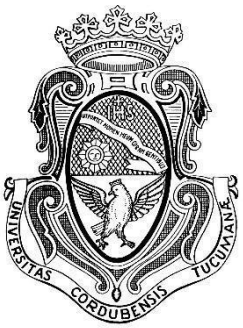
FCEFyN Facultad de Ciencias Exactas Físicas y Naturales



UNIVERSIDAD NACIONAL DE CÓRDOBA

FACULTAD DE CIENCIAS EXACTAS, FÍSICAS Y NATURALES

PROGRAMACIÓN CONCURRENTE Y PARALELA

Trabajo Practico 1

Grupo Thread-ators

Alumnos:

Santiago Martin Macagno
Pedro Caldera
Luciano Gabriel Bogni
Ignacio Hernán Garay
Lucio Alejandro Espinoza

Repositorio: https://github.com/santi-macagno/tp1-pcyp-2026

Índice

1. Introducción
2. Descripción del problema
3. Arquitectura de la solución
4. Recursos compartidos
5. Condiciones de carrera
6. Estrategia de sincronización
7. Condiciones de espera y despertar
8. Mecanismo de terminación
9. Preservación de los invariantes
10. Análisis teórico del tiempo de ejecución
11. Resultados experimentales
12. Comparación entre tiempos teóricos y observados
13. Efecto de la cantidad de hilos y de las demoras
14. Justificación de las decisiones de diseño
15. Estrategia de pruebas
16. Conclusiones
Anexo A. Compilación y ejecución
Anexo B. Archivos de salida

1. Introducción

El objetivo del trabajo es desarrollar en Java 21 un sistema concurrente que simule una granja de impresión 3D. El sistema recibe órdenes de fabricación, les asigna una impresora, valida los modelos, ejecuta las impresiones y controla la calidad de las piezas. Las cuatro etapas funcionan de forma concurrente y comparten recursos: colas de órdenes, un conjunto de impresoras y un registro de eventos. Todas las órdenes deben procesarse conservando la consistencia de sus estados, y el programa debe terminar sin dejar hilos activos.

Este informe describe la arquitectura de la solución, identifica los recursos compartidos y las condiciones de carrera posibles, explica la estrategia de sincronización y el mecanismo de terminación.

La sincronización se implementa exclusivamente con synchronized, Object.wait() y Object.notifyAll(), creando y esperando los hilos con Thread.

2. Descripción del problema

2.1 Modelo

La granja contiene una **matriz de impresoras** (10 filas × 20 columnas en la configuración oficial). Cada impresora tiene un identificador único, que la solución forma como P-fila-columna, una posición, un estado, un contador de usos y la orden asignada, si la hay. Sus estados posibles son AVAILABLE (disponible), RESERVED (reservada por una orden en validación o impresión) y OUT_OF_SERVICE (fuera de servicio y no reutilizable).

Las **órdenes** se identifican de 1 a totalOrders, empiezan en CREATED y terminan en exactamente uno de cuatro estados terminales: APPROVED, REJECTED, PRINT_FAILED o DEFECTIVE. Los estados intermedios son WAITING_VALIDATION, READY_TO_PRINT y PRINTED.

2.2 Etapas y transiciones

Etapa	Hilos (oficial)	Transición de la orden	Efecto sobre la impresora	Demora
1. Asignación	3	CREATED → WAITING_VALIDATION	AVAILABLE → RESERVED; usos + 1	50 ms
2. Validación	2	válido: → READY_TO_PRINT inválido: → REJECTED	válido: sigue RESERVED inválido: → AVAILABLE	120 ms
3. Impresión	3	éxito: → PRINTED falla: → PRINT_FAILED	éxito: → AVAILABLE falla: → OUT_OF_SERVICE	180 ms
4. Control de calidad	2	→ APPROVED o → DEFECTIVE	ninguno (ya fue liberada)	80 ms

2.3 Determinismo de los resultados

Las decisiones de validación, impresión y calidad se toman con la clase provista OutcomeDecider. Cada decisión es una función pura de la semilla, el identificador de la orden y una constante distinta por etapa: combina esos valores con una función hash y compara el resultado módulo 100 con el porcentaje configurado. Por eso los estados finales no dependen del orden en que se ejecuten los hilos. Con la configuración oficial son siempre **359 APPROVED, 71 REJECTED, 45 PRINT_FAILED y 25 DEFECTIVE**. Lo que puede variar entre ejecuciones es la impresora asignada a cada orden, los contadores de cada impresora, los tiempos y el orden global de los eventos.

2.4 Configuración

Main lee config/tp1.properties, y Simulation.execute(config) usa solo la configuración que recibe, sin volver a leer el archivo. Ninguna cantidad, probabilidad, tiempo ni ruta está escrita en el código. La configuración

oficial define 500 órdenes; una matriz de 10 × 20 impresoras; 3, 2, 3 y 2 hilos para las cuatro etapas; 85 % de modelos válidos, 90 % de impresiones exitosas y 95 % de piezas aprobadas; y demoras de 50, 120, 180 y 80 ms.

2.5 Restricciones

Solo se permiten Thread, Runnable, synchronized, wait/notify/notifyAll, Lock, ReentrantLock, Semaphore, los ejecutores básicos y las colecciones comunes no thread-safe. Están prohibidos BlockingQueue, las colecciones concurrentes, Condition, los latches y barreras, las clases atómicas, Collections.synchronized*, los streams paralelos y la espera activa. Las demoras representan la duración de las operaciones y no pueden usarse como mecanismo de sincronización. Tampoco se permite terminar los hilos con Thread.stop ni con System.exit.

3. Arquitectura de la solución

3.1 Visión general

La solución es un **pipeline de productores y consumidores encadenados**: cada etapa consume órdenes de una cola y las produce en la siguiente. A eso se suma un **pool de recursos compartido**, la granja de impresoras, que usan tres de las cuatro etapas. Las colas y el pool son monitores. Todos los hilos registran sus transiciones en un único registro de eventos.

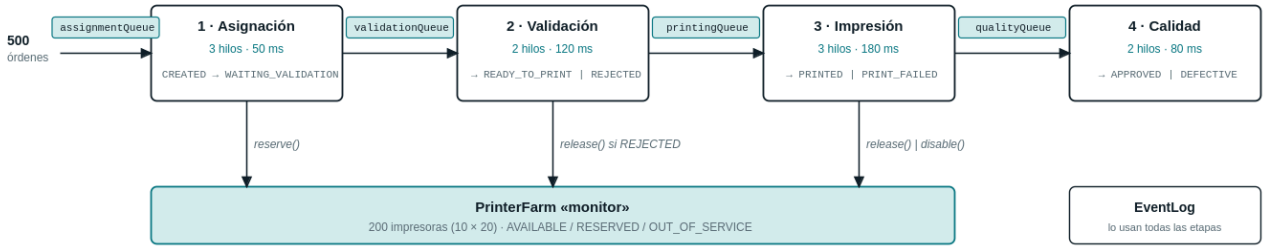


Figura 1. Pipeline de etapas, colas y recursos compartidos.

3.2 Organización en paquetes

Paquete	Clases	Responsabilidad
api (cátedra)	Simulation, SimulationConfig, SimulationResult, OutcomeDecider, snapshots y enums	Contrato con la cátedra. No se modifica.
solution	ConcurrentSimulation	Orquestación: crea los recursos, arranca los hilos, coordina la terminación y arma el resultado.
solution.model	Order, Printer, Stage	Datos del dominio.
solution.monitor	OrderQueue, PrinterFarm	Monitores: objetos compartidos que hacen esperar y despertar hilos.
solution.log	EventLog	Transiciones de las órdenes y filas de eventos.csv.
solution.output	ResultWriter	Escritura de los tres archivos de resultados.
solution.workers	stageWorkers y sus cuatro subclases	Lógica de cada etapa (Runnable).

3.3 Diagrama de clases

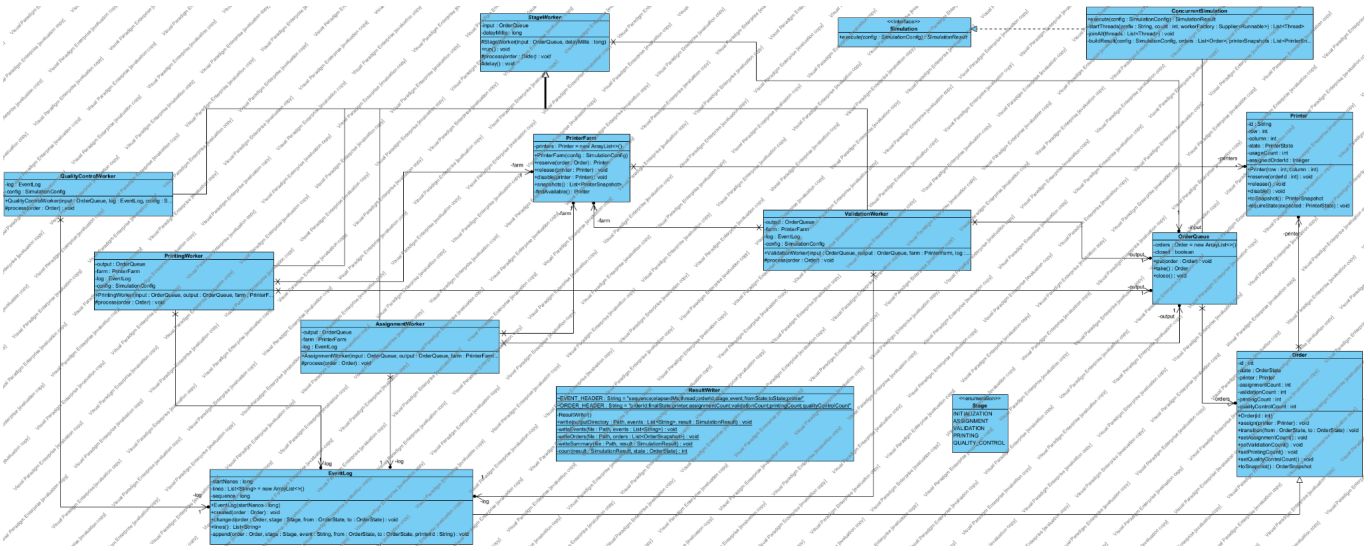


Figura 2. Diagrama de clases de la solución.

Descripción de cada clase:

- **Order**: estado, impresora histórica y cuatro contadores, uno por etapa. Sus métodos son synchronized, salvo getId(), que lee un campo final. transition(from, to) lanza una excepción si el estado actual no es from, lo que detecta cualquier doble procesamiento. assign() falla si la orden ya tenía una impresora.
- **Printer**: no es thread-safe a propósito, porque sus métodos se invocan siempre con el lock de PrinterFarm tomado. Cada método valida el estado de origen: reserve exige AVAILABLE, y release y disable exigen RESERVED. assignedOrderId es un Integer para poder valer null cuando la impresora está libre.
- **Stage**: enumeración de las etapas que se escriben en la columna stage del registro.
- **OrderQueue**: monitor que encapsula un ArrayList y un indicador closed. Ofrece put, take (bloqueante, devuelve null cuando la cola está cerrada y vacía) y close.
- **PrinterFarm**: monitor que encapsula las impresoras. Ofrece reserve (bloqueante mientras no haya impresoras disponibles), release, disable y snapshots.
- **EventLog**: guarda en memoria las filas de eventos.csv. changed() realiza la transición de la orden y escribe su fila dentro de la misma sección crítica.
- **ResultWriter**: escribe los tres archivos desde el hilo principal, cuando ya terminaron todos los workers.
- **stageWorkers**: clase base abstracta que sigue el patrón *template method*. Su run() (final) repite take() → process() hasta que la cola devuelve null. Cada subclase implementa process() para su etapa.
- **ConcurrentSimulation**: implementa Simulation. Crea los recursos compartidos, arranca todos los hilos con nombres descriptivos (`assignment-1`, `validation-2`, etc.), coordina la terminación en cascada y construye el `SimulationResult`.

4. Recursos compartidos

Recurso	Hilos que lo acceden	Mecanismo de protección
assignmentQueue	main (put y close), 3 de asignación (take)	lock intrínseco de la OrderQueue
validationQueue	3 de asignación (put), 2 de validación (take), main (close)	lock intrínseco de la OrderQueue
printingQueue	2 de validación (put), 3 de impresión (take), main	lock intrínseco de la OrderQueue

	(close)	
qualityQueue	3 de impresión (put), 2 de calidad (take), main (close)	lock intrínseco de la OrderQueue
PrinterFarm y sus impresoras	asignación (reserve), validación (release), impresión (release y disable), main (snapshots)	lock intrínseco de la PrinterFarm, que protege a todas las Printer
EventLog (secuencia y filas)	los 10 workers y main	lock intrínseco del EventLog
Cada Order	un hilo por vez, a medida que avanza por las etapas; main al final	lock propio de la orden, más el traspaso por las colas y join()
SimulationConfig, OutcomeDecider	todos	ninguno: un record inmutable y una función pura

5. Condiciones de carrera

Sin sincronización, podrían ocurrir las siguientes condiciones de carrera. Para cada una se indica el mecanismo que la impide.

1. **Doble extracción de una orden.** Dos validadores ven la misma orden al frente de la cola y la extraen los dos. La orden se valida dos veces, lo que viola el invariante 3, o el ArrayDeque queda en un estado inconsistente. *Lo impide* que take() sea synchronized.
2. **Pérdida de una orden.** Dos productores agregan a la vez y una inserción pisa a la otra. Una orden nunca llega a la etapa siguiente, lo que viola los invariantes 2 y 9. *Lo impide* que put() sea synchronized.
3. **Doble reserva de una impresora** (*check-then-act*). Dos asignadores ven libre la misma impresora antes de que alguno la marque como reservada. Viola el invariante 4. *Lo impide* que la búsqueda y la reserva estén en la misma sección crítica de PrinterFarm.reserve().
4. **Incremento perdido del contador de usos.** usageCount++ es una lectura, una suma y una escritura: dos hilos pueden leer el mismo valor y registrar un solo uso. Viola el invariante 8. *Lo impide* que el incremento ocurra dentro del lock de la farm.
5. **Secuencia duplicada o desordenada.** Dos hilos leen el mismo valor de sequence, u obtienen sus números en un orden y escriben sus filas en el otro. *Lo impide* que el incremento, la medición del tiempo y el agregado de la fila ocurran en la misma sección crítica.
6. **Fila registrada antes que la transición anterior.** Si una orden se encolara antes de registrar su transición, el consumidor podría procesarla mientras sigue en su estado anterior, o registrar su fila antes. *Lo impide* el orden fijo **registrar → liberar → encolar**.
7. **Impresora aparentemente asignada a dos órdenes en el registro.** Si se liberara la impresora antes de registrar el estado terminal de la orden, un asignador podría reservarla y registrar su asignación primero. *Lo impide* el mismo orden.
8. **Lectura de valores obsoletos.** Sin una relación *happens-before*, el hilo que recibe una orden podría ver un estado anterior guardado en caché. *Lo impiden* el traspaso por los monitores y join().
9. **Señal perdida** (*lost wake-up*). Si un consumidor evaluara la condición fuera del lock y después llamara a wait(), un put() o un close() intermedio no lo despertaría. *Lo impide* que la evaluación y la espera ocurran dentro del mismo método synchronized.

6. Estrategia de sincronización

6.1 Monitores propios

Toda la exclusión mutua y la espera por condición está encapsulada en dos monitores, construidos con el lock intrínseco de cada objeto y su única cola de espera. Las tres reglas que siguen son:

- **wait() siempre dentro de un while.** Java implementa semántica Mesa: el hilo notificado compite por el lock y, cuando lo obtiene, la condición puede haber dejado de cumplirse. Además, la especificación admite despertares espurios.
- **notifyAll() y no notify().** Cada objeto tiene una sola cola de espera, y Condition no está permitida. Despertar a todos garantiza que ningún aviso se pierda; cada hilo vuelve a evaluar su condición.
- **Secciones críticas mínimas.** Nunca se llama a Thread.sleep() ni se hace entrada/salida con un lock tomado, porque sleep() no libera el monitor.

```
// OrderQueue
public synchronized Order take() throws InterruptedException {
    while (orders.isEmpty() && !closed) {
        wait();
    }
     if (orders.isEmpty()) {
        return null;   // cerrada y vacia
    }
    return orders.removeFirst();

}

// PrinterFarm: buscar y reservar en la misma seccion critica
public synchronized Printer reserve(Order order) throws InterruptedException {
    Printer printer = findAvailable();
    while (printer == null) {
        wait();
        printer = findAvailable();
    }
    printer.reserve(order.getId());
    return printer;
}
```

6.2 Workers sin sincronización propia y traspaso de propiedad

Los workers no tienen ningún bloque synchronized ni llamadas a wait o notify: solo invocan a los monitores en el orden correcto. Cada orden pertenece a un único hilo por vez, y las colas son los puntos de traspaso entre etapas. La demora de cada etapa se aplica una vez por orden, fuera de cualquier lock, así que las demoras de distintos hilos ocurren realmente en paralelo.

6.3 Orden de las operaciones en cada etapa

Cada etapa sigue el orden **registrar → liberar la impresora → encolar a la siguiente etapa**. Registrar antes de encolar garantiza que el consumidor reciba la orden ya en su nuevo estado y con su fila escrita. Registrar antes de liberar garantiza que en el registro una impresora nunca aparezca asignada a dos órdenes al mismo tiempo.

```
// printingWorker.process(...)
delay();
order.setPrintingCount();
if (OutcomeDecider.isPrintSuccessful(order.getId(), config)) {
    // 1. registrar  2. liberar  3. encolar
    log.changed(order, Stage.PRINTING, READY_TO_PRINT, PRINTED);
    farm.release(order.getPrinter());
    output.put(order);
} else {
    log.changed(order, Stage.PRINTING, READY_TO_PRINT, PRINT_FAILED);
    farm.disable(order.getPrinter());
}
```

6.4 Atomicidad de la transición y su registro, y orden de los locks

EventLog.changed() cambia el estado de la orden y escribe su fila dentro de la misma sección crítica. Así, el orden de las filas coincide exactamente con el orden en que se confirman las transiciones, como pide el enunciado. Por eso los workers no invocan order.transition() por su cuenta. Esta decisión introduce el único anidamiento de locks del sistema: con el lock del registro tomado se adquiere el de la orden. El

anidamiento ocurre siempre en el sentido **registro → orden**, y ningún método sincronizado de Order invoca al registro, de modo que nunca puede formarse una espera circular.

6.5 Visibilidad

El modelo de memoria de Java garantiza una relación *happens-before* entre la liberación de un lock y la siguiente adquisición del mismo lock. Esa relación hace visible el estado de una orden entre el productor y el consumidor de cada cola. Thread.join() hace visible para el hilo principal todo lo que hicieron los workers, y los campos final (identificadores, startNanos y las referencias de los workers) son visibles sin sincronización después de la construcción.

6.6 Ausencia de interbloqueo

No hay espera circular por tres motivos. Primero, los monitores de colas y de impresoras nunca adquieren otro lock mientras tienen el propio, y ningún worker invoca un monitor desde dentro de otro, así que no puede ocurrir *nested monitor lockout*. Segundo, el único anidamiento sigue siempre el mismo orden. Tercero, las esperas por condición forman una cadena hacia adelante: cada etapa espera a la anterior, y la asignación espera impresoras que liberan la validación y la impresión, que nunca esperan a la asignación. Como el enunciado garantiza que nunca se agotan las impresoras operativas, toda espera en reserve() termina.

7. Condiciones de espera y despertar

Hilo que espera	Dónde	Condición de espera	Quién lo despierta
Cualquier worker	OrderQueue.take()	la cola está vacía y abierta	put() de la etapa anterior (main, en la primera cola) o close() desde main
Asignación	PrinterFarm.reserve()	no hay ninguna impresora AVAILABLE	release(): validación al rechazar o impresión al terminar con éxito
main	Thread.join()	el hilo sigue vivo	la finalización del hilo
Cualquiera	entrada a un método synchronized	otro hilo tiene el lock	la salida de ese hilo del método

disable() no despierta a ningún hilo porque dar de baja una impresora no aumenta la cantidad disponible. Thread.sleep() no es una espera de sincronización: modela la duración de cada operación.

8. Mecanismo de terminación

La terminación se basa en **cerrar las colas en cascada**, sin *poison pills*, sin Thread.stop y sin System.exit. El hilo principal crea las 500 órdenes, las coloca en la cola de asignación y la cierra. Después arranca simultáneamente todos los hilos de las cuatro etapas. Luego espera con join() a los hilos de cada etapa y cierra la cola siguiente:

```
joinAll(assignment);   validationQueue.close();
joinAll(validation);   printingQueue.close();
joinAll(printing);     qualityQueue.close();
joinAll(quality);
```

Un worker termina cuando take() devuelve null, lo que ocurre solo cuando su cola está cerrada y vacía. Como cada cola se cierra recién después de que terminaron todos sus productores, en ese momento ya contiene todo lo que va a recibir: ninguna orden se pierde. close() invoca notifyAll(), así que los consumidores bloqueados despiertan y terminan. Cerrar una cola antes de tiempo provocaría que sus consumidores terminen mientras todavía faltan órdenes, y que los productores fallen al encolar.

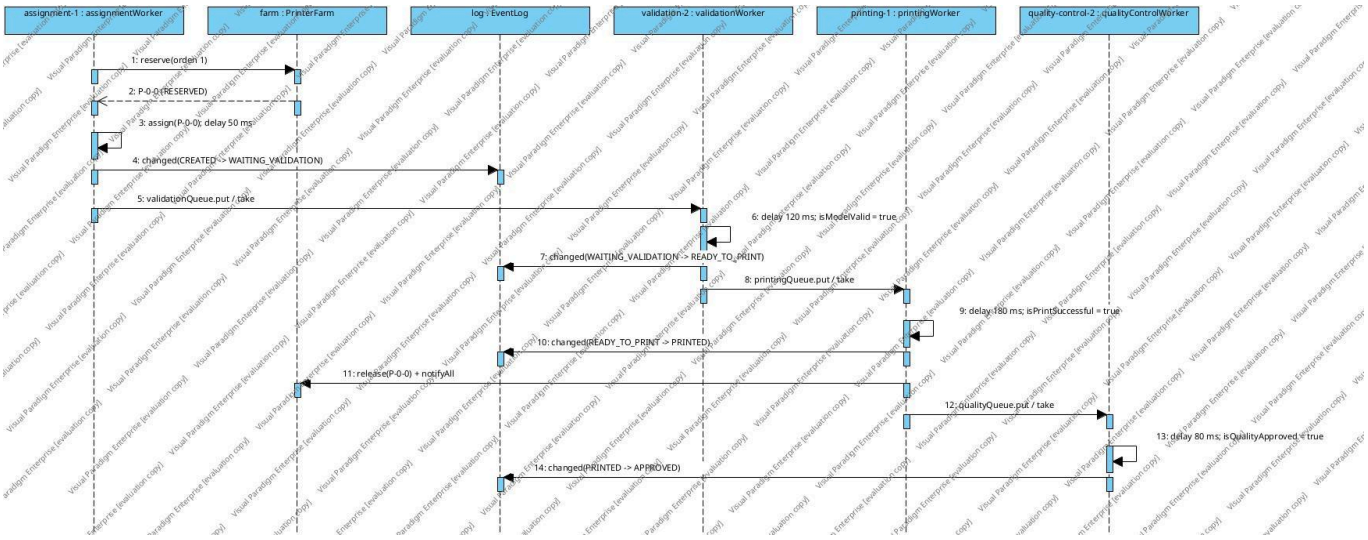


Figura 3. Diagrama de secuencia de la terminación en cascada.

Después del último join(), allThreadsTerminated se calcula verificando isAlive() en cada hilo, y luego se construye el resultado y se escriben los archivos. Si un worker terminara por una excepción, su join() retornaría y la cascada continuaría, así que la simulación no se bloquea. Las órdenes que hubieran quedado en estados intermedios serían detectadas por las pruebas de invariantes.

9. Preservación de los invariantes

#	Invariante	Cómo lo garantiza la solución
1	Cada orden existe en un solo estado a la vez.	Un único campo de estado protegido por el lock de la orden; transition(from, to) valida el estado actual.
2	Cada orden alcanza exactamente un estado terminal.	Ninguna transición sale de un estado terminal, y la terminación en cascada vacía todas las colas.
3	Ninguna orden se procesa más veces de lo establecido.	Cada orden está en una sola cola por vez; take() la entrega a un único hilo; transition() rechaza repeticiones.
4	Una impresora no se asigna a dos órdenes a la vez.	Búsqueda y reserva en la misma sección crítica; Printer.reserve() exige AVAILABLE.
5	Toda orden en WAITING_VALIDATION o READY_TO_PRINT tiene una impresora reservada.	La impresora se reserva antes de pasar a WAITING_VALIDATION y se libera recién al salir de READY_TO_PRINT.
6	Una orden terminal no retiene una impresora.	REJECTED → release(); PRINTED → release(); PRINT_FAILED → disable().
7	Una impresora OUT_OF_SERVICE no se reutiliza.	disable() la retira; findAvailable() solo devuelve impresoras AVAILABLE; release() sobre una impresora rota lanza una excepción.
8	El contador de usos coincide con la cantidad de asignaciones.	usageCount++ ocurre dentro de Printer.reserve(), siempre con el lock de la farm.
9	Al final no quedan órdenes en estados intermedios.	Cada cola se cierra solo cuando terminaron sus productores, y sus consumidores la vacían.
10	Al final no quedan impresoras RESERVED.	Consecuencia de los invariantes 6 y 9.

10. Análisis teórico del tiempo de ejecución

10.1 Modelo

En un pipeline, el tiempo total lo determina la etapa de menor capacidad, el **cuello de botella**, más la latencia necesaria para que la primera orden llegue a esa etapa y la última salga del sistema. Para cada etapa *k*:

$$T_k = N_k \cdot d_k / h_k \text{ y } T_{total} \approx max_k(T_k) + \Sigma_k d_k$$

donde N_k es la cantidad de órdenes que procesa la etapa, d_k su demora y h_k su cantidad de hilos. La suma Σ dk incluye solo las demoras de las etapas distintas del cuello de botella: las anteriores retrasan la llegada de la primera orden al cuello y las posteriores la salida de la última. Con la configuración oficial, la validación procesa las 500 órdenes, la impresión las 429 válidas y el control de calidad las 384 impresas con éxito.

Etapa	N	d	h	T teórico	Capacidad
Asignación	500	50 ms	3	8,33 s	60,0 órdenes/s
Validación	500	120 ms	2	**30,00 s**	16,7 órdenes/s
Impresión	429	180 ms	3	25,74 s	16,7 órdenes/s
Control de calidad	384	80 ms	2	15,36 s	25,0 órdenes/s

El cuello de botella es la validación. La latencia es la suma de las demoras de las demás etapas: 50 ms de asignación antes del cuello, y 180 + 80 ms de impresión y control de calidad después, en total 310 ms. La demora de la validación no se suma aparte porque ya está incluida en sus 30,00 s. El tiempo teórico es entonces **30,00 + 0,31 ≈ 30,31 s**, dentro del rango de 30 a 45 s que exige el enunciado.

10.2 Saturación del pool de impresoras

Cada orden retiene una impresora desde la asignación hasta que es rechazada o termina de imprimirse. La asignación produce unas 60 órdenes/s, pero las impresoras se liberan al ritmo del cuello de botella, unas 16,7/s. La cantidad de impresoras retenidas crece entonces a una tasa de 60 − 16,7 ≈ 43,3 por segundo, y el pool de 200 se agota en aproximadamente 200 / 43,3 ≈ **4,6 s**. A partir de ese momento los hilos de asignación quedan bloqueados en reserve() y avanzan al ritmo de las liberaciones. Por eso la asignación no termina en 8,33 s sino junto con el avance de la validación: el pool actúa como un **buffer acotado natural** (*backpressure*), y por eso las colas no necesitan un límite de capacidad.

11. Resultados experimentales

11.1 Metodología

Las ejecuciones se realizaron sobre el código de la rama de desarrollo, empaquetado con mvn clean package y ejecutado con java -jar target/tp1.jar, en una computadora con un AMD Ryzen 7 5800x y 24 GB de RAM, Linux 7.2 y OpenJDK 21.0.12. Para cada variante se modificó únicamente config/tp1.properties. De cada ejecución se tomó durationMillis de resumen.properties, y de eventos.csv se obtuvieron el instante de la última transición de cada etapa y el primer instante en que no queda ninguna impresora disponible, es decir, cuando las reservadas más las fuera de servicio suman 200.

Se realizaron **22 ejecuciones en 6 configuraciones**. En **todas** los estados finales fueron exactamente 359 / 71 / 45 / 25, con allThreadsTerminated=true y remainingIntermediateOrders=0.

11.2 Configuración oficial

Ejecución	Duración (ms)	Pool agotado (ms)	Máx. reservadas
1	30 430	4 449	196
2	30 427	4 443	196
3	30 429	4 446	196
4	30 427	4 444	196
5	30 429	4 447	196
Promedio	**30 428**	**4 446**	**196**

La desviación entre ejecuciones es de alrededor de 1 ms. En promedio, las etapas terminan a los 20,25 s (asignación), 30,11 s (validación), 30,35 s (impresión) y 30,43 s (control de calidad). Cada hilo de validación procesó exactamente 250 órdenes.

11.3 Ocupación de impresoras

Entre 0 y 4,45 s, el llenado: las reservadas crecen unas 43 por segundo, como predice el modelo. Entre 4,45 y 20,25 s, la saturación: no queda ninguna impresora disponible, y las reservadas descienden lentamente a medida que otras pasan a fuera de servicio. Desde 20,25 s, el vaciado: la asignación terminó y las impresoras vuelven al pool. Al final no queda ninguna reservada y hay 45 fuera de servicio, una por cada orden PRINT_FAILED.

11.4 Variantes

Configuración (hilos asig/val/imp/cal)	Ejecuciones	Promedio	Mín. – máx.	Pool agotado	Fin de la asignación
Oficial (3/2/3/2)	5	30,43 s	30,43 – 30,43 s	4,45 s	20,25 s
3 hilos de validación (3/3/3/2)	5	26,12 s	26,12 – 26,12 s	4,82 s	16,75 s
3 de validación + 4 de impresión (3/3/4/2)	3	20,46 s	20,46 – 20,46 s	5,25 s	13,65 s
6 hilos de asignación (6/2/3/2)	3	30,35 s	30,31 – 30,37 s	1,94 s	20,25 s
1 hilo por etapa (1/1/1/1)	3	77,55 s	77,55 – 77,56 s	14,56 s	49,43 s
Demoras duplicadas (3/2/3/2)	3	60,80 s	60,80 – 60,80 s	8,85 s	40,45 s

12. Comparación entre tiempos teóricos y observados

Configuración	Cuello de botella	Teórico	Observado	Diferencia
Oficial	validación: 500 × 120 / 2 = 30,00 s	30,31 s	30,43 s	+0,12 s
3 hilos de validación	impresión: 429 × 180 / 3 = 25,74 s	25,99 s	26,12 s	+0,13 s
3 val + 4 imp	validación: 500 × 120 / 3 = 20,00 s	20,31 s	20,46 s	+0,15 s
6 hilos de asignación	validación: 30,00 s	30,31 s	30,35 s	+0,04 s
1 hilo por etapa	impresión: 429 × 180 / 1 = 77,22 s	77,47 s	77,55 s	+0,08 s
Demoras duplicadas	validación: 500 × 240 / 2 = 60,00 s	60,62 s	60,80 s	+0,18 s

En las seis configuraciones el tiempo observado supera al teórico entre 0,04 y 0,18 s, menos del 0,8 % del total. Que la diferencia sea siempre positiva es lo esperable, porque el modelo no incluye costos que solo pueden sumar tiempo: crear y arrancar los hilos, despertar a los que esperan en wait(), la imprecisión de Thread.sleep() y la escritura de los archivos. La predicción también se cumple para la saturación del pool: con 6 hilos de asignación, la tasa neta de ocupación es de 120 − 16,7 ≈ 103 impresoras por segundo, y el modelo predice que el pool se agota en 200 / 103 ≈ 1,94 s, exactamente lo medido. La variación entre ejecuciones es muy baja (de 1 a 2 ms, y 29 ms con 6 asignadores) porque el tiempo está dominado por las demoras, frente a las cuales el costo de los locks y de la planificación de hilos es despreciable.

13. Efecto de la cantidad de hilos y de las demoras

- **Solo ayuda agregar hilos en el cuello de botella.** Al pasar de 2 a 3 validadores, el tiempo baja de 30,43 a 26,12 s y el cuello se desplaza a la impresión. Al agregar un cuarto hilo de impresión, el cuello vuelve a la validación, ahora con 20 s, y el total baja a 20,46 s: una reducción del 33 % respecto de la configuración oficial.
- **Agregar hilos fuera del cuello de botella no mejora el tiempo.** Duplicar los hilos de asignación (de 3 a 6) deja el total prácticamente igual (30,35 s). Lo único que cambia es que el pool se satura antes (1,94 s contra 4,45 s), y la asignación sigue terminando a los 20,25 s porque está limitada por la devolución de impresoras.
- **Con un hilo por etapa**, el cuello pasa a ser la impresión, la etapa con mayor trabajo por orden (429 × 180 ms), y el tiempo sube a 77,55 s. Esto muestra que el paralelismo dentro de cada etapa es lo que permite alcanzar el rango requerido.
- **Las demoras escalan el tiempo de forma lineal.** Al duplicarlas, el total pasa de 30,43 a 60,80 s (factor 1,998). Aumentar la demora de una etapa que no es cuello de botella no tiene efecto hasta que su tiempo supera al del cuello. Por ejemplo, el control de calidad debería pasar de 80 a unos 160 ms para igualar a la validación.
- **Con demoras nulas**, el tiempo queda dominado por la sincronización y la creación de hilos. Las pruebas de invariantes ejecutan 141 simulaciones completas en alrededor de 2 s.
- **La cantidad de impresoras** cambia el momento de la saturación pero no el tiempo total, mientras no sea tan baja que limite el trabajo en vuelo de las etapas de validación e impresión.

14. Justificación de las decisiones de diseño

Decisión	Alternativa considerada	Justificación
Monitores con synchronized y wait/notifyAll	Semaphore o ReentrantLock	Es el mecanismo que mejor expresa las condiciones de espera; queda encapsulado en dos clases y no requiere llevar la cuenta de permisos a mano.
Colas sin límite de capacidad	Buffer acotado (por ejemplo, un arreglo circular)	El pool de impresoras ya limita el trabajo en vuelo (backpressure). Un buffer acotado agregaría una segunda condición de espera sobre la misma cola de espera del monitor.
ArrayList como estructura interna	ArrayDeque, LinkedList, arreglo propio	Colección estándar y simple de usar, como pide el enunciado. removeFirst() desplaza los elementos restantes (O(n)), mientras que ArrayDeque lo haría en O(1); con a lo sumo 500 órdenes por cola, ese costo es despreciable frente a las demoras de cada etapa.
Cierre de cola y retorno null	Poison pills	No requiere conocer la cantidad de consumidores ni

		crear órdenes ficticias; close() despierta a todos los consumidores de una vez.
Terminación en cascada desde el hilo principal	Que el último worker de cada etapa cierre la siguiente	No necesita un contador compartido adicional (CountDownLatch está prohibido) y su razonamiento es lineal.
Transición y registro atómicos en EventLog	Transicionar en el worker y registrar después	Garantiza que el orden de las filas sea el orden de confirmación. Su costo es un único anidamiento de locks, con orden fijo.
Eventos en memoria, escritos al final	Escritura incremental del archivo	Evita la entrada/salida dentro de secciones críticas; el volumen (unas 2300 filas) es pequeño.
Búsqueda lineal de impresoras	Cola de impresoras disponibles	Más simple; con 200 impresoras el costo es despreciable, y el uso resultó repartido (de 1 a 5 usos por impresora).
Thread en lugar de ExecutorService	Ejecutores	Es el camino que recomienda el enunciado, permite nombrar los hilos y deja explícita la terminación.
Crear todas las órdenes antes de arrancar los hilos	Crearlas mientras corren las etapas	La creación no es una etapa y dura milisegundos; todas las etapas arrancan simultáneamente.

15. Estrategia de pruebas

`mvn clean test` ejecuta 40 pruebas: 7 públicas y 33 desarrolladas por el grupo. Todas pasan con la versión concurrente.

Prueba	Ejecuciones	Qué verifica
Pruebas públicas (5 clases)	7	Lectura de la configuración, determinismo de OutcomeDecider, resultados y contadores, reutilización de una única impresora, formato de los archivos y restricciones del código fuente.
OrderQueueTest	12	Con 3 productores y 4 consumidores, 3000 órdenes se transfieren sin pérdidas ni duplicados (10 repeticiones); close() despierta a los consumidores bloqueados; una cola cerrada entrega las órdenes que le quedaban.
PrinterFarmTest	13	Creación de la matriz con sus identificadores; 8 hilos realizan 1600 reservas sobre 3 impresoras sin asignaciones simultáneas y con contadores exactos (10 repeticiones); una impresora reutilizada por 8 órdenes con espera; una impresora fuera de servicio nunca se reasigna.
InvariantsTest + InvariantChecker	8	7 escenarios × 20 ejecuciones (oficial sin demoras, una impresora, pocas impresoras con muchos hilos, un hilo por etapa, todo rechazado, todas las impresiones fallan, demoras pequeñas) más la configuración oficial sin demoras: 141 simulaciones con tiempo límite para detectar interbloqueos. En cada una se verifican los 10 invariantes recorriendo eventos.csv evento por evento.

Las pruebas públicas también pasaban con una versión secuencial preliminar, sin ningún hilo. Por eso la evidencia de que la sincronización es correcta la aportan las pruebas propias, que usan varios hilos y repeticiones.

16. Conclusiones

La solución procesa todas las órdenes, preserva los diez invariantes en las 141 simulaciones de prueba y en las 22 ejecuciones medidas, y termina sin dejar hilos activos. Toda la sincronización queda

concentrada en dos monitores y en el registro de eventos, lo que mantiene a los workers libres de lógica de sincronización y facilita razonar sobre la ausencia de carreras y de interbloqueos.

El modelo de pipeline predijo los tiempos de las seis configuraciones con diferencias de entre 0,04 y 0,18 s (menos del 0,8 %), siempre con el tiempo observado levemente por encima del teórico. Los experimentos confirman que el rendimiento lo determina exclusivamente el cuello de botella, que agregar hilos fuera de él no mejora el tiempo, y que el pool de impresoras actúa como mecanismo de contención entre la asignación y las etapas más lentas.

Anexo A. Compilación y ejecución

```
mvn clean test              # ejecuta las pruebas
mvn clean package           # genera target/tp1.jar
java -jar target/tp1.jar    # ejecuta con config/tp1.properties; resultados en resultados/
```

Anexo B. Archivos de salida

eventos.csv: una fila por cambio de estado de una orden (2313 filas en la configuración oficial).

```
sequence;elapsedMs;thread;orderId;stage;event;fromState;toState;printer
1;13;main;1;INITIALIZATION;ORDER_CREATED;;CREATED;
502;87;assignment-1;1;ASSIGNMENT;ORDER_STATE_CHANGED;CREATED;WAITING_VALIDATION;P-0-0
510;208;validation-2;1;VALIDATION;ORDER_STATE_CHANGED;WAITING_VALIDATION;READY_TO_PRINT;P-0-0
526;388;printing-1;1;PRINTING;ORDER_STATE_CHANGED;READY_TO_PRINT;PRINTED;P-0-0
534;468;quality-control-2;1;QUALITY_CONTROL;ORDER_STATE_CHANGED;PRINTED;APPROVED;P-0-0
```

Las filas corresponden a la orden 1 en una ejecución oficial: la diferencia entre ellas coincide con las demoras de cada etapa (120, 180 y 80 ms).

elementos.csv: una fila por orden, ordenada por identificador.

```
orderId;finalState;printer;assignmentCount;validationCount;printingCount;qualityControlCount
1;APPROVED;P-0-0;1;1;1;1
7;REJECTED;P-0-6;1;1;0;0
```

resumen.properties de una ejecución oficial:

```
totalOrders=500
processedOrders=500
approvedOrders=359
rejectedOrders=71
printFailedOrders=45
defectiveOrders=25
durationMillis=30429
allThreadsTerminated=true
remainingIntermediateOrders=0
```